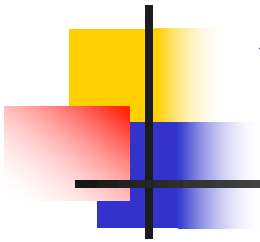


言語処理系 第3回
下向き構文解析

木村啓二
keiji@waseda.jp

第1回演習問題1解答

- <名前> → [a-zA-Z]([a-zA-Z]|[0-9])*に関して以下の問に答えよ
  - <名前>にマッチする文字列を以下から選べ
    - Compiler1
    - compiler@2
    - 2026cs
  - ただし、[a-z]はaからzまでの英語アルファベット、[A-Z]はAからZまでのアルファベット、[a-zA-Z]は大文字小文字全てのアルファベット（<英字>と表記する）、[0-9]は0から9までの数字（<数字>と表記する）を意味する
  - Compiler1

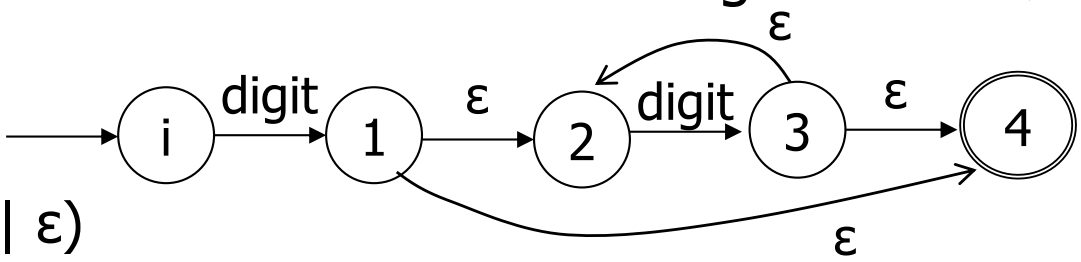


第2回演習問題2

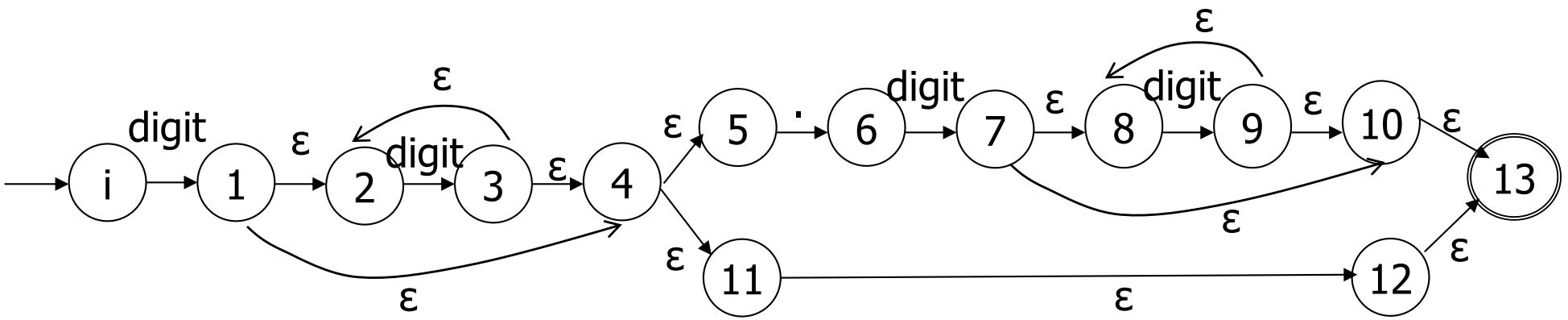
- 以下の正規表現に対応するNFAを作成せよ。次にDFAを作成せよ。ただし、digitは0-9の数字一文字である。
  - digit digit*
  - digit digit* (. digit digit* | ε)

第2回演習問題2解答(1)

- 以下の正規表現に対応するNFAを作成せよ。ただし、digitは0-9の数字一文字である。
  - digit digit*

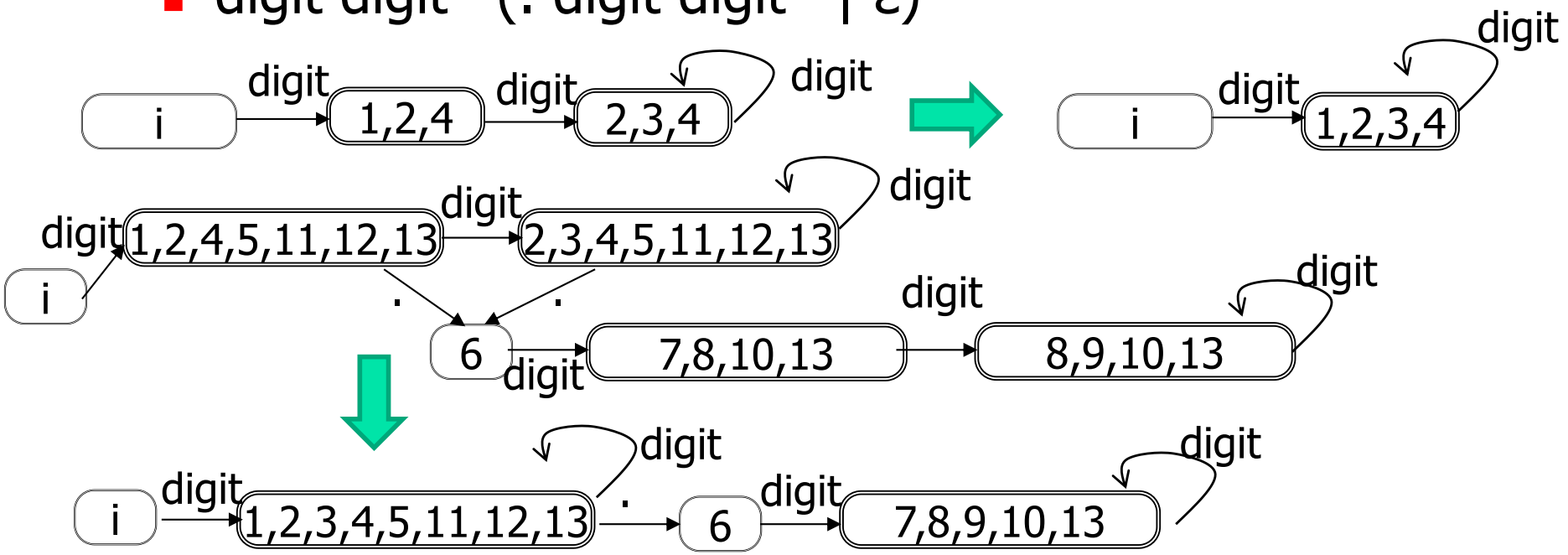


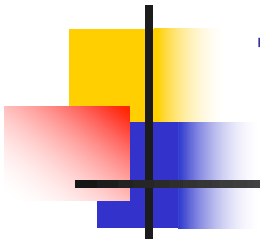
  - digit digit* (. digit digit* | ε)



第2回演習問題2解答(2)

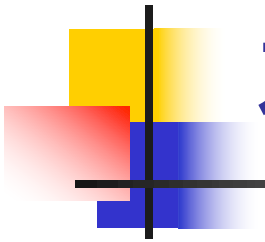
- DFAを作成せよ
  - digit digit*
  - digit digit* (. digit digit* | ε)





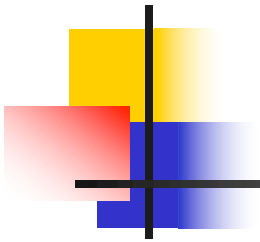
前回のおさらい

- 正規表現により字句を定義する
  - 集合の定義を行う
    - 記号の集合を組み合わせる(演算する)ことで、目当ての字句(記号列)の集合を定義する
- 正規表現より字句を読み取る状態遷移機械を作る
  - 入力となる記号列中の個々の記号が、字句定義中の各集合の要素かどうかチェックする



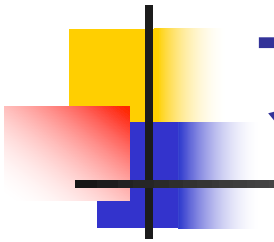
本日の内容

- プログラミング言語の文法
  - 文脈自由文法
    - 生成規則により定義される文法
- 下向き構文解析
  - 下向き構文解析適用可能な文法
    - LL(1)
  - コード例



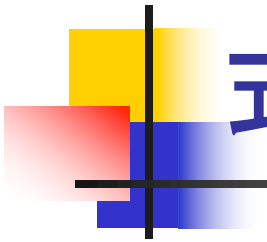
字句解析・構文解析

- コンパイラのフロントエンド部の処理
- ソースプログラムはファイル上の単なる文字列
  - 文字列から「書いてあること」を解析する必要がある
- 字句解析
  - ソースプログラムから単語を切り出す
- 構文解析
  - 式の解析が大きな動機の一つ
  - 字句解析で切り出した単語のつながりから、構文規則(syntax)をチェックする
    - 字句間のつながり方（式の場合、演算順序も表現する）をチェックする
  - さらにプログラムを内部表現に落とし込む



プログラミング言語の文法（例）

- プログラムを構成する要素の集合と、各要素を構成する記号列並びの定義だと考えることが出来る
- 要素の中に自分自身を含められる（再帰的な定義が出来る）と便利
  - 実行文
    - { 実行文; … ; 実行文; }
    - if (式) 実行文
    - if (式) 実行文 else 実行文
    - for (式; 式; 式;) 実行文
    - 代入文;
  - 代入文
    - 変数 = 式 （左辺はかなり簡略化している）
  - 式
    - 式+項
  - 項
    - 項*数値
    - 項*変数



式の計算(評価)順序の解釈は？

- 以下の式が入力として与えられたとする
  - a+b*c
- 人間はb*cを先に計算することがすぐにわかる
- 機械はa, +, b, *, cと順番に読むしかない
  - 実際には字句解析結果の記号の順番として読み出す
    - 名前, +, 名前, *, 名前
- a, +まで読み取って、次にbを読み取ったときに、a+bを計算して良いか？
  - 当然駄目。ではどうする？
  - 1) bの後に*を読み取るかもしれないという前提で記号を読む
  - 2) bの後に読み取る可能性のある記号列をいくつか考慮して記号を読む

文脈自由文法 (context free grammar)

- 言語は記号列の組み合わせ規則と考える
  - <加算式>は<式>+<式>から成り立つ、式は...
  - <加算式>→<式>+<式>（生成規則）
    - 一般にはU→x
  - 言語を**単純な記号列の再帰的な組み合わせと見なす**
- 文脈自由文法
  - 文脈自由文法Gは生成規則の集合Pと記号Sの組として定義される G={P,S}（Sは開始記号）
  - G={P,S}においてPの生成規則の左辺に現れる記号を非終端記号、右辺にだけ現れる記号を終端記号
  - 文章の意味までは問わない（生成規則を満たしていれば、非終端記号の順番までは問わない）
- 生成規則の記号の集合を語彙(vocabulary)と呼ぶ

再帰的な構造を扱えることが正規表現との大きな違い

文脈自由文法の例

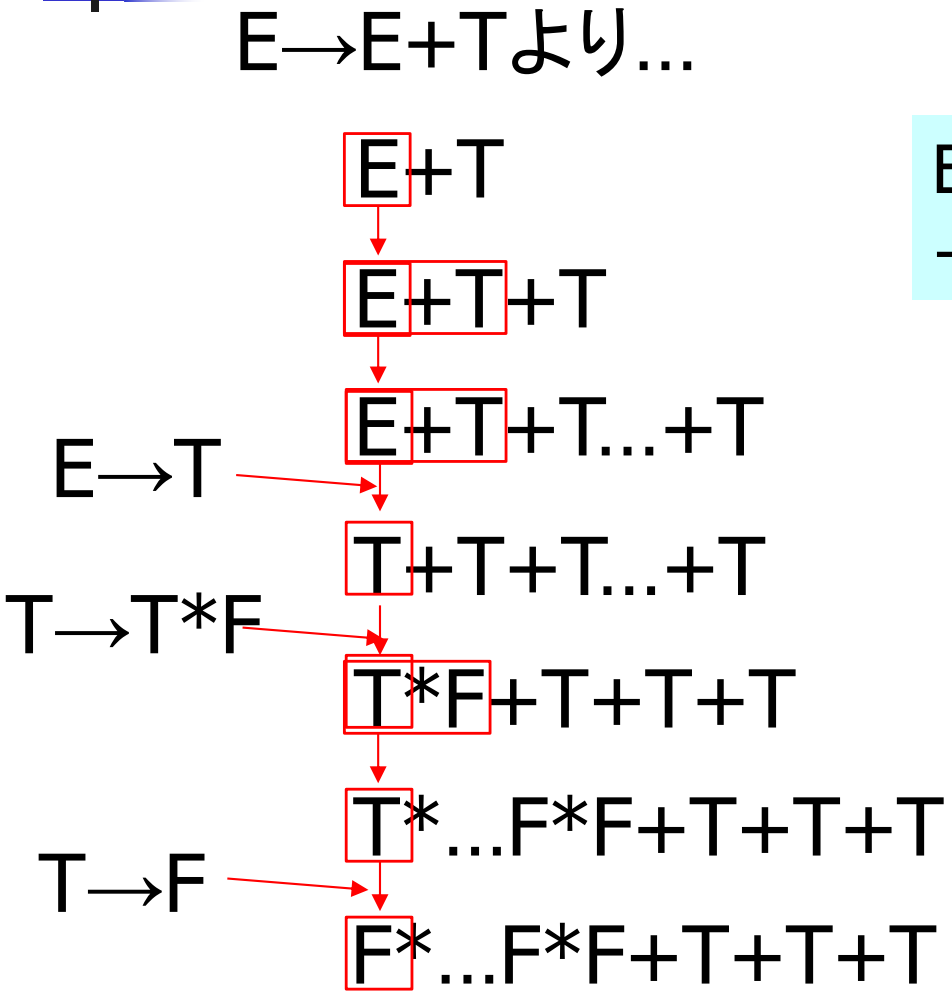
- G={P,E}
- P={E→E+T
  E→T
  T→T*F
  T→F
  F→(E)
  F→<名前>
  F→<数字>

語彙Vは
V={E,T,F,+,*,(,),<名前>,<数字>}

+,*,(,),<名前>,<数字>は
終端記号

演算の優先順位も自然に決まる:
Tは*で繋がれた項の連続、
Eは+で繋がれたTの項の連続
→*は+より優先度が高い

再帰的な定義の効果



Eは最終的に任意の数のTを
+で結んだものである

Tは最終的に任意の数のFを
*で結んだものである

導出と還元

- 文法G={P,S}において、x,y∈V*（語彙の任意の並び），U→u∈Pのとき、v,w∈V*が
  v=xUy, w=xuy
  ならばvはwを直接導出（v⇒w）
- v,w∈V*においてvから生成規則を複数回適用しwが得られたとき($v=u_0⇒u_1⇒u_2⇒\cdots⇒u_n⇒w$)
  - vはwを**導出**
  - wはvに**還元**
  - n>=0ならば$v\overset{*}{⇒}w$
- 例：先の文法G={P,E}において、
  E ⇒T ⇒F ⇒<数字>と、Eは<数字>を導出（<数字>はEに還元）
  E ⇒T ⇒T*F ⇒<名前>*F ⇒<名前>*<数字>と、Eは<名前>*<数字>を導出（<名前>*<数字>はEに還元）

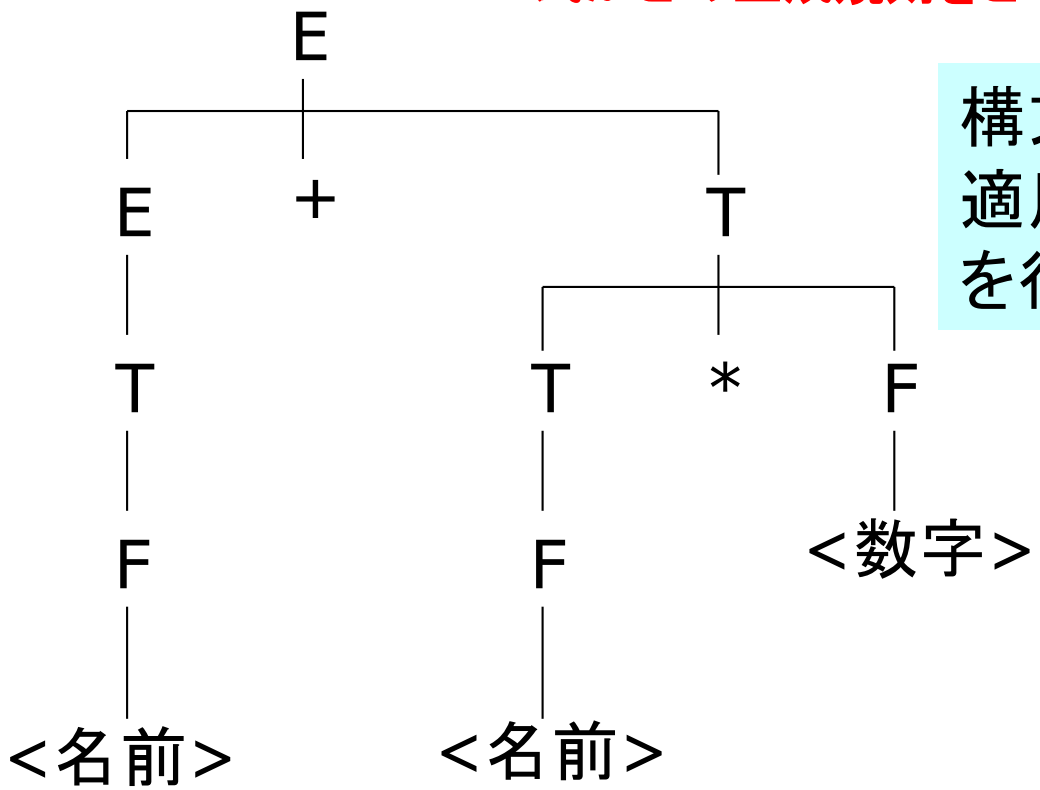
構文解析の役割

- 終端記号の列を受け取り、その列が文法Gの開始記号Sからどのように導出できるかを解析する
  - 終端記号は字句解析器から受け取る
- 例：先の文法G={P,E}と入力”a+b*3”
  - 字句解析からは”<名前>+<名前>*<数字>”を受け取る
  - E ⇒ E+T
⇒ T+T（E→Tより）
⇒ F+T(T→Fより)
⇒ <名前>+T(F→<名前>より)
⇒ <名前>+T*F(T→T*Fより)
⇒ <名前>+F*F(T→Fより)
⇒ <名前>+<名前>*F(F→<名前>より)
⇒ <名前>+<名前>*<数字>(F→<数字>より)

解析木

- 根は開始記号、葉は終端記号、中間ノードは非終端記号
- <名前>+<名前>*<数字>の解析木

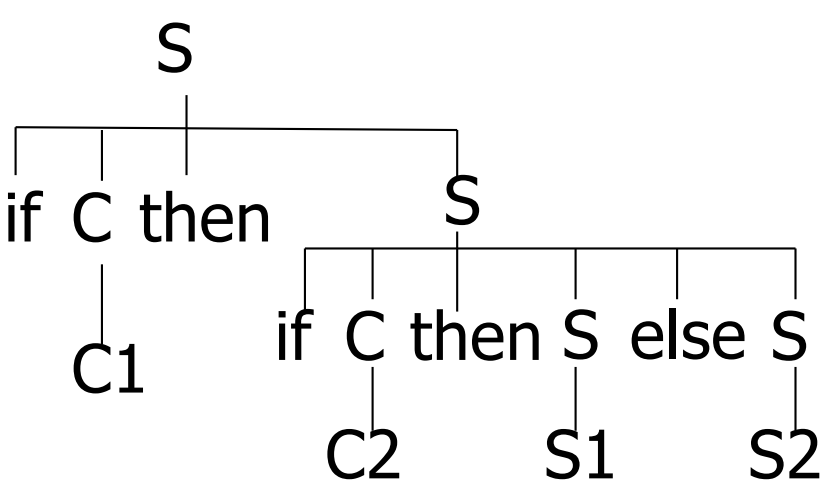
式がどの生成規則をどのように適用できる構成か

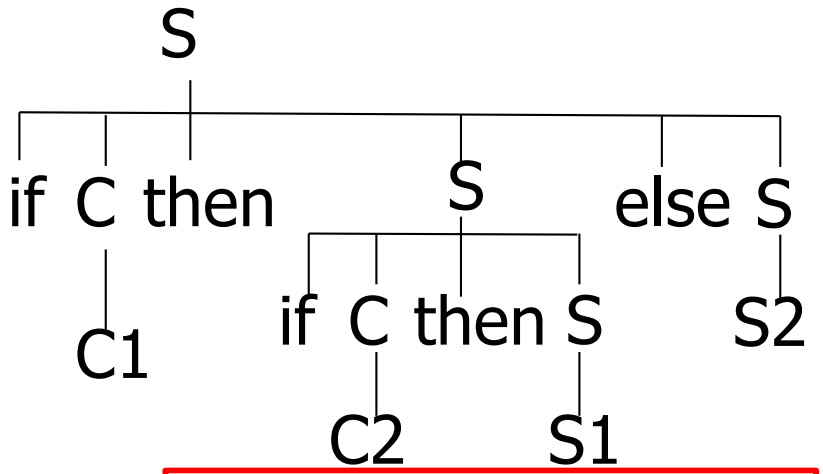


構文解析は生成規則の適用により文の解析を行う

曖昧な文法

- 解析木が一意に決定できない文が出現しうる文法
- 例:
  - S → if C then S
  - S → if C then S else S
    - if C1 then if C2 then S1 else S2の構文木は？





優先順位や付帯規則で対処

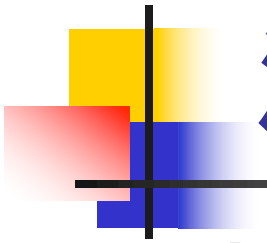
最左導出と最右導出
<数字>+<名前>*<数字>を例に

- 最左導出
  - 最も左の非終端記号に生成規則を適用
  - E⇒E+T
⇒T+T
⇒F+T
⇒<数字>+T
⇒<数字>+T*F
⇒<数字>+F*F
⇒<数字>+<名前>*F
⇒<数字>+<名前>*<数字>
- 最右導出
  - 最も右の非終端記号に生成規則を適用
  - E⇒E+T
⇒E+T*F
⇒E+T*<数字>
⇒E+F*<数字>
⇒E+<名前>*<数字>
⇒T+<名前>*<数字>
⇒ F+<名前>*<数字>
⇒ <数字>+<名前>*<数字>

入力記号列から考えると、
入力に対して生成規則を(ほぼ一意に)
当てはめていって、文法に
適合するかどうか確認する戦略

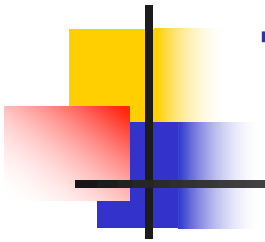
次回分まで学習してから改めて
考え直してみてください

入力記号列から考えると、
入力に対して当てはまる生成規則を
試していって、最終的に文法に
適合するかどうか確認する戦略



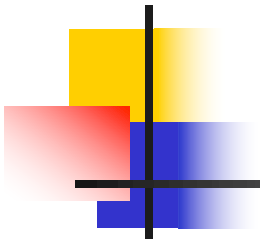
演習問題1

- (i+j)*kに先に示した文法Gの生成規則を適用して解析木を構築する。Gの開始記号はEである。これに関して以下の問いに答えよ。
- 1. 上記式に対して開始記号に最初に適用できる生成規則は何か
- 2. 2番目に適用できる生成規則は何か
- 3. (i+j)の導出ではF→(E)を適用するが、2番目の生成規則とF→(E)の間に適用する生成規則を答えよ



下向き構文解析

- 構文木を上から下に構築していく
- 手作りしやすい
  - 生成規則に対応する関数を実装すればよい
- 最左導出によるアルゴリズム
- 文法がLL(1)であれば下向き構文解析を適用可能



LL(1)文法

- 左から右へトークンを一つ先読みしつつ最左導出できる文法のクラス
- おおまかな条件
  - $A \rightarrow \alpha | \beta$ において、一つのトークンを先読みすることでαとβの選択ができる
  - Left-to-right scanning, 1 symbol Lookahead

下向き構文解析のプログラムの様子

S → aBd (a,dは終端記号、Bは非終端記号)

```
void S()
{
   aを読む
   B()
   dを読む
}
```

言語は生成規則の再帰的な組み合わせなので，再帰関数呼び出しにより解析する

どの順番で記号が出現するか決め打ちしながら解析を進める

B → b|bc

```
void B()
{
   bを読む
      または
   bを読む
   cを読む
}
```

B → b(c|ε)
ただし，εは記号がない記号であり，終端記号

括りだし

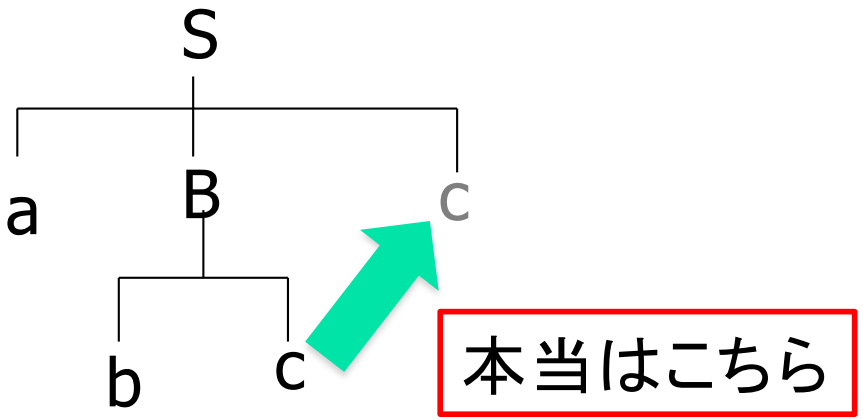


```
void B()
{
   bを読む
   {
      cを読む
         または
      何も読まない
   }
}
```

括りだしもできない例

S → aBc
B → b|bc　(B → b(c|ε))

入力がabcだった場合は



Sの処理でBに後戻りして「実はεを読んだ」とやり直す
（がこんなことはやりたくない）

左再帰性

E → E+T

```
void E()
{
   E()
   +を読む
   T()
}
```

E()が無限に呼び出される

Eの生成規則の左辺がEで始まらないように書き換える
（方法は後で説明）

LL(1)文法の定義

- 文法G={V_N,V_T,P,S}及び記号列$\alpha \in (V_N \cup V_T)^*$と$A \in V_N$に関してFirst($\alpha$), Follow(A)を定義
  - ただしV_N,V_Tはそれぞれ非終端記号、終端記号の集合
  - $First(\alpha) = \{a | a \in V_T, \alpha \overset{*}{\Rightarrow} a \cdots\}$
    ただし$\alpha \overset{*}{\Rightarrow} \varepsilon$なら$\varepsilon \in First(\alpha)$
  - $Follow(A) = \{a | a \in V_T, S \overset{*}{\Rightarrow} \cdots Aa \cdots\}$

αが終端記号なら
First(α)はα

開始記号SのFollowには入力終了を表す特別な終端記号「$」も入る

- さらにA→αがあるときにDirector(A, α)を定義

$$Director(A, \alpha) = \{a | a \in V_T, a \in First(\alpha) \vee \left(\alpha \overset{*}{\Rightarrow} \varepsilon \wedge a \in Follow(A)\right)\}$$

- このとき文法Gにおいて、任意の$A \in V_N$とAを左辺に持つ生成規則$A \rightarrow \alpha_1 | \alpha_2 | \cdots | \alpha_n$に対して以下の条件を満たせばGはLL(1)である

$$Director(A, \alpha_i) \cap Director(A, \alpha_j) = \emptyset, i \neq j$$

先のスライドの条件の直感的な解釈

- Directorの意味
  - A→ α において,
    - αの先頭で読み出しうる終端記号
    - αが空ならAの次に読み出しうる終端記号
- A→ α , A→ β において, Director(A, α), Director(A, β)に重複が無いとは…
  - 生成規則Aの処理を行う際に
    最初の終端記号(トークン)を読めば
    迷うことなく導出できる

文法G（資料12枚目）の場合の例（<名前>と<数字>は共にiで表す）

- First(E)=First(T)=First(F)={i,(}
  First(i)={i}, First(()={(}, First())={)}
  First(+)={+}, First(*)={*}
  - First(E+T)とか考えられるがこの文法ではそれは First(E)と等価（$E \overset{*}{\Rightarrow} \varepsilon$がないから）
  - 記号列αで考えるのはDirectorで使うから
- Follow(E)={+,),$}（$は終了記号）
  Follow(T)=Follow(F)={+,*,),$}
- Director(E,E+T)=Director(E,T)
  =Director(T,T*F)=Director(T,F)={i,(}
  Director(F,(E))={(},Director(F,i)={i}

FirstとFollowに関するヒント

- First(T)
  - Tを左辺とする生成規則を基点とする
  - 右辺の左端が終端記号ならFirst(T)に追加
  - 非終端記号ならそれを左辺とする生成規則に対して上記を繰り返す
- Follow(T)
  - 開始記号SのFollow(S)には$を加える
  - Tを右辺に持つ生成規則を探し，そのTの右側をチェックする
    - Tの右が終端記号ならFollow(T)に追加
    - Tの右が非終端記号ならそれのFirstをFollow(T)に追加
    - ただし，上記Firstがεかつその右側に記号があれば
それのFirstをFollow(T)に追加
  - Tが右辺の右端もしくは右側記号が右端かつそのFirstがεなら、左辺の記号に対して上記を繰り返しFollow(T)に追加
    - Follow(F)の場合、T→Fがあるのでに対して上記を繰り返す
- FollowはFirstにεが入っていたときに使う
  - Followはεを含まない．Directorもεを含まない．

文法Gにおける Follow(T)を考える

- G={P,E}
- P={E→**E**+T
  E→T
  T→**T***F
  T→F
  F→(**E**)
  F→<名前>
  F→<数字>

TのFollowはEのFollowでもある

*は自明

先の文法G={P,E}の書き換え

- 生成規則の右辺の先頭が左辺と同一だと困る
  - A→Aα|β の場合、右辺のAの処理はどうする？
    - ただし、β はAから始まらない
- 以下のルールで書き換える
  - A→Aα|β はA→βA’, A’→αA’|ε
    - βに複数のαが続く
  - A→Aα1|Aα2|…|Aαm|β1|β2|…|βnは
    A→β1A’|β2A’|…|βnA’, A’→α1A’|α2A’|…|αnA’|ε
- 先の文法Gは以下G2になる
  - E→TE′, E′→+TE′|ε
    T→FT′, T′→*FT′|ε
    F→(E)|<数字>|<名前>
  - 元の文法のE→E+T|Tから、Tをくくりだして E→TE′、E′→=+TE′|εとする（Eの導出を続けると、いずれTを導出することを思い出す）

再帰的下向き構文解析プログラム

```
void E()
{
  T();
  while (token == SYM_PLUS) {
    token = gettoken();
    T();
  }
}
void T()
{
  F();
  while (token == SYM_MULT) {
    token = gettoken();
    F();
  }
}
```

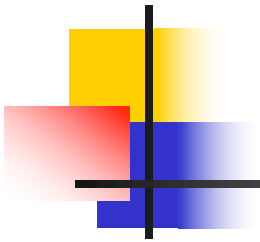
実際には、解析しつつ計算や
次回以降で説明する
中間表現の作成を行う

文法G2をさらに
以下のように整理
E→T{+T}
（{+T}は+Tの
0回以上の繰り返し）
T→F{*F}
（{*F}は*Fの
0回以上の繰り返し）
F→(E)|<数字>|<名前>

```
void F()
{
  if (token == SYM_LEFTPAR) { // "("
    token = gettoken();
    E();
    if (token == SYM_RIGHTPAR) { // ")"
      token = gettoken();
    } else {
      error();
    }
  } else if (token == <名前> || token == <数字>) {
    token = gettoken();
  } else {
    error();
  }
}
```

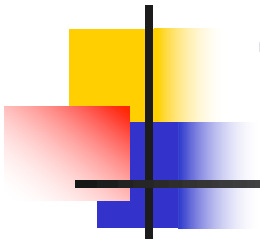
例:<名前>+<名前>*<数字>の解析

- E()
  - T()
    - F()
      - F()中でtokenが<名前>なのでgettoken()で次の字句(+)を読み取りreturn
    - T()中でtokenがSYM_MULT(*)ではないのでreturn
    - ここまででE→T{+T}のTとT→F{*F}のFとF→<名前>の導出ができた
  - E()中でtokenがSYM_PLUSなのでgettoken()で次の字句(<名前>)を読み出しT()(E→T{+T}の+Tの導出)
    - F()
      - F()中でtokenが<名前>なのでgettoken()で次の字句(*)を読み取りreturn
      - T→F{*F}のFの導出とF→<名前>の導出
    - T()中でtokenがSYM_MULT(*)なのでgettoken()で次の字句(<数字>)を読み出しF()(T→F{*F}の*Fの導出)
      - F()中でtokenが<数字>なのでgettoken()で次の字句(*)を読み取りreturn



あいまいな文法の扱い

- 以下はLL(1)ではない
  - S → if C then S …(1)
  - S → if C then S else S …(1)
    - else∈Follow(S)
- 次の入力がelseなら(2)の生成規則を選ぶ、というように特別扱いをする



まとめ

- 構文解析の概要
  - 構文木
  - 下向き構文解析
- 実際に自分で再帰下降構文解析器を書いてみてください

演習問題2



- 資料30ページの文法G2のFirst, Follow, Directorをそれぞれ示せ（<名前>と<数字>は共にiで表す）
  - First(E), First(E’), First(T), First(T’), First(F)
    - 非終端記号のFirstは自明なので略
  - Follow(E), Follow(E’), Follow(T), Follow(T’), Follow(F)
  - Director(E,TE’),
    Director(E’,+TE’), Director(E’, ε),
    Director(T,FT’),
    Director(T’,*FT’), Director(T’,ε),
    Director(F,(E)), Director(F,i)
  - Moodle上では一部だけ答えるように設定するが、各自で実際に一度は全て求めることを推奨する